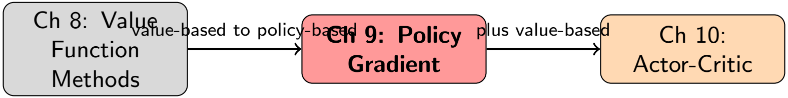
Chapter 9: Policy Gradient Methods

Adapted from Shiyu Zhao's TextBook

Outline

Where Policy Gradient Methods Fit



Key Transition: From value-based methods to policy-based methods

- Previous chapters: policies represented by **tables**
- This chapter: policies represented by **parameterized functions** $\pi(a|s, \theta)$
- Optimal policies obtained by optimizing scalar metrics via gradient ascent

Advantages: More efficient for large state/action spaces, stronger generalization, handles continuous actions naturally

Chapter Overview

Basic idea of policy gradient:

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta J(\theta_t)$$

where $J(\theta)$ is a scalar metric and $\nabla_\theta J$ is its gradient.

Three key questions to answer:

1. **What metrics should be used?** (Section 9.2)
   - Average state value $\bar{v}_\pi$
   - Average reward $\bar{r}_\pi$
2. **How to calculate the gradients?** (Section 9.3)
   - Policy Gradient Theorem
   - Discounted and undiscounted cases
3. **How to use samples to estimate gradients?** (Section 9.4)
   - REINFORCE algorithm

Tabular Policy Representation

Tabular representation: Store action probabilities for all states in a table

	a_1	a_2	a_3	a_4	a_5
s_1	$\pi(a_1 \vert s_1)$	$\pi(a_2 \vert s_1)$	$\pi(a_3 \vert s_1)$	$\pi(a_4 \vert s_1)$	$\pi(a_5 \vert s_1)$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
s_9	$\pi(a_1 \vert s_9)$	$\pi(a_2 \vert s_9)$	$\pi(a_3 \vert s_9)$	$\pi(a_4 \vert s_9)$	$\pi(a_5 \vert s_9)$

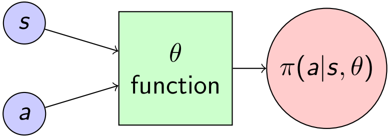
Properties:

- **Define optimal:** Maximize every state value
- **Update policy:** Directly change entries in table
- **Retrieve probability:** Look up corresponding entry
- Problem: Inefficient for large state/action spaces

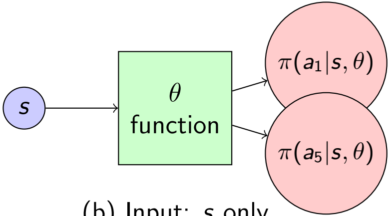
Function Representation of Policies

Function representation: Policy parameterized by $\theta \in \mathbb{R}^m$

$$\pi(a|s,\theta) \quad \text{or equivalently} \quad \pi_\theta(a|s)$$



(a) Input: (s, a)



(b) Input: s only

Key differences from tabular:

- **Define optimal:** Maximize certain **scalar metrics**
- **Update policy:** Change parameter θ (cannot directly change probabilities)

Softmax Policy

Common parameterization: Softmax function ensures valid probabilities

Softmax Policy

$$\pi(a|s,\theta) = \frac{e^{h(s,a,\theta)}}{\sum_{a' \in \mathcal{A}} e^{h(s,a',\theta)}}, \quad a \in \mathcal{A}$$

where $h(s,a,\theta)$ indicates the preference for selecting a at s.

Properties:

- $\pi(a|s,\theta) \in (0,1)$ for all (s,a) — always positive!
- $\sum_{a \in \mathcal{A}} \pi(a|s,\theta) = 1$ — valid probability distribution
- Policy is **stochastic** and hence **exploratory**
- Can be realized by a neural network with softmax output layer

Important: $\pi(a|s,\theta) > 0$ ensures $\ln \pi(a|s,\theta)$ is well-defined (needed for policy gradient)

Metric 1: Average State Value

Average State Value

$$\bar{v}_\pi = \sum_{s \in \mathcal{S}} d(s) v_\pi(s) = \mathbb{E}_{S \sim d}[v_\pi(S)]$$

where $d(s)$ is the weight/distribution of state s.

Two cases for choosing d:

Case 1: $d = d_0$ independent of π (denoted $\bar{v}_\pi^0$)

- Uniform: $d_0(s) = 1/|\mathcal{S}|$ (all states equally important)
- Single start state s_0: $d_0(s_0) = 1$, $d_0(s \neq s_0) = 0$

Case 2: $d = d_\pi$ is the **stationary distribution** under π

- Satisfies $d_\pi^T P_\pi = d_\pi^T$
- Reflects long-term behavior: frequently visited states get higher weights

Equivalent Expressions of $\bar{v}_\pi$

Expression 1: Weighted sum

$$\bar{v}_\pi = \sum_{s\in\mathcal{S}} d(s) v_\pi(s)$$

Expression 2: Expected discounted return

$$J(\theta) = \lim_{n\to\infty} \mathbb{E}\left[\sum_{t=0}^{n} \gamma^t R_{t+1}\right] = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1}\right]$$

Proof of equivalence:

$$\begin{aligned}\mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1}\right] &= \sum_{s\in\mathcal{S}} d(s)\mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \middle| S_0 = s\right] \\ &= \sum_{s\in\mathcal{S}} d(s) v_\pi(s) = \bar{v}_\pi\end{aligned}$$

Expression 3: Inner product form

$$\bar{v}_\pi = d^T v_\pi$$

Metric 2: Average Reward

Average Reward

$$\bar{r}_\pi = \sum_{s \in \mathcal{S}} d_\pi(s) r_\pi(s) = \mathbb{E}_{S \sim d_\pi}[r_\pi(S)]$$

where d_π is the stationary distribution and

$$r_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s, \theta) r(s, a) = \mathbb{E}_{A \sim \pi(s,\theta)}[r(s, A)|s]$$

is the expected immediate reward at state s.

Equivalent expression: Average reward per step

$$J(\theta) = \lim_{n \to \infty} \frac{1}{n} \mathbb{E}\left[\sum_{t=0}^{n-1} R_{t+1}\right] = \bar{r}_\pi$$

Inner product form:

$$\bar{r}_\pi = d_\pi^T r_\pi$$

Equivalence of Metrics in Discounted Case

Lemma (Equivalence between $\bar{v}_\pi$ and $\bar{r}_\pi$)

In the discounted case where $\gamma \in (0, 1)$:

$$\bar{r}_\pi = (1 - \gamma)\bar{v}_\pi$$

Proof:

- We have $\bar{v}_\pi = d_\pi^T v_\pi$ and $\bar{r}_\pi = d_\pi^T r_\pi$
- Bellman equation: $v_\pi = r_\pi + \gamma P_\pi v_\pi$
- Multiply d_π^T on both sides:

$$\bar{v}_\pi = \bar{r}_\pi + \gamma d_\pi^T P_\pi v_\pi = \bar{r}_\pi + \gamma d_\pi^T v_\pi = \bar{r}_\pi + \gamma \bar{v}_\pi$$

  (using $d_\pi^T P_\pi = d_\pi^T$)
- Rearranging: $\bar{v}_\pi - \gamma \bar{v}_\pi = \bar{r}_\pi \Rightarrow \bar{r}_\pi = (1 - \gamma)\bar{v}_\pi$

Implication: These metrics can be **simultaneously maximized**!

Summary of Metrics

Metric	Expression 1	Expression 2	Expression 3
$\bar{v}_\pi$	$\sum_s d(s) v_\pi(s)$	$\mathbb{E}_{S \sim d}[v_\pi(S)]$	$\lim_{n \to \infty} \mathbb{E}\left[\sum_{t=0}^{n} \gamma^t R_{t+1}\right]$
$\bar{r}_\pi$	$\sum_s d_\pi(s) r_\pi(s)$	$\mathbb{E}_{S \sim d_\pi}[r_\pi(S)]$	$\lim_{n \to \infty} \frac{1}{n} \mathbb{E}\left[\sum_{t=0}^{n-1} R_{t+1}\right]$

Notation:

- $\bar{v}_\pi$: average state value with $d = d_\pi$ (stationary distribution)
- $\bar{v}_\pi^0$: average state value with $d = d_0$ (independent of π)
- $\bar{r}_\pi$: average reward

Goal: Find optimal θ to maximize these metrics via gradient ascent:

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta J(\theta_t)$$

Policy Gradient Theorem

Theorem (Policy Gradient Theorem)

The gradient of $J(\theta)$ is:

$$\nabla_\theta J(\theta) = \sum_{s \in \mathcal{S}} \eta(s) \sum_{a \in \mathcal{A}} \nabla_\theta \pi(a|s,\theta) q_\pi(s,a)$$

where η is a state distribution. This has a compact form:

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}_{S \sim \eta, A \sim \pi(S,\theta)} \left[\nabla_\theta \ln \pi(A|S,\theta) \cdot q_\pi(S,A)\right]}$$

Important remarks:

- This theorem summarizes results for different metrics ($\bar{v}_\pi^0$, $\bar{v}_\pi$, $\bar{r}_\pi$)
- The specific form of $J(\theta)$ and η varies by scenario
- The compact form is expressed as an **expectation** — enables stochastic gradient

Question: Why can we write $\nabla_\theta \pi$ as $\pi \cdot \nabla_\theta \ln \pi$?

Key identity:

$$\nabla_\theta \ln \pi(a|s,\theta) = \frac{\nabla_\theta \pi(a|s,\theta)}{\pi(a|s,\theta)}$$

Therefore:

$$\nabla_\theta \pi(a|s,\theta) = \pi(a|s,\theta)\nabla_\theta \ln \pi(a|s,\theta)$$

Derivation of compact form:

$$\begin{aligned}
\nabla_\theta J(\theta) &= \sum_s \eta(s) \sum_a \nabla_\theta \pi(a|s,\theta) q_\pi(s,a) \\
&= \sum_s \eta(s) \sum_a \pi(a|s,\theta)\nabla_\theta \ln \pi(a|s,\theta) q_\pi(s,a) \\
&= \mathbb{E}_{S\sim\eta}\left[\sum_a \pi(a|S,\theta)\nabla_\theta \ln \pi(a|S,\theta) q_\pi(S,a)\right] \\
&= \mathbb{E}_{S\sim\eta, A\sim\pi(S,\theta)}\left[\nabla_\theta \ln \pi(A|S,\theta) q_\pi(S,A)\right]
\end{aligned}$$

From True Gradient to Stochastic Gradient

True gradient (gradient ascent):

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta J(\theta_t) = \theta_t + \alpha \mathbb{E}\left[\nabla_\theta \ln \pi(A|S, \theta_t) q_\pi(S, A)\right]$$

Problem: True gradient involves an expectation — cannot compute exactly

Solution: Use **stochastic gradient** (sample-based approximation)

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t, \theta_t) q_t(s_t, a_t)$$

where $q_t(s_t, a_t)$ is an approximation of $q_\pi(s_t, a_t)$.

How to estimate $q_\pi(s_t, a_t)$?

- **Monte Carlo:** Use actual return $q_t(s_t, a_t) = G_t = \sum_{k=t+1}^{T} \gamma^{k-t-1} r_k$
- This gives the **REINFORCE** algorithm

REINFORCE Algorithm

Algorithm: REINFORCE (Monte Carlo Policy Gradient)

Initialize: parameter θ, discount $\gamma \in (0, 1)$, learning rate $\alpha > 0$
Goal: Learn optimal policy maximizing $J(\theta)$

For each episode:

1. Generate episode $\{s_0, a_0, r_1, s_1, a_1, r_2, \ldots, s_{T-1}, a_{T-1}, r_T\}$ following $\pi(\theta)$
2. **For** $t = 0, 1, \ldots, T-1$:
    - **Value estimate:** $q_t(s_t, a_t) = \sum_{k=t+1}^{T} \gamma^{k-t-1} r_k$
    - **Policy update:** $\theta \leftarrow \theta + \alpha \nabla_\theta \ln \pi(a_t | s_t, \theta) \cdot q_t(s_t, a_t)$

Key features:

- Uses Monte Carlo return to estimate q_π
- On-policy: samples generated by current policy
- Simple but can have high variance

Interpretation of REINFORCE Update

Rewrite the update: Using $\nabla_\theta \ln \pi = \frac{\nabla_\theta \pi}{\pi}$:

$$\theta_{t+1} = \theta_t + \alpha \frac{q_t(s_t, a_t)}{\pi(a_t|s_t, \theta_t)} \nabla_\theta \pi(a_t|s_t, \theta_t)$$

Let $\beta_t = \frac{q_t(s_t, a_t)}{\pi(a_t|s_t, \theta_t)}$. Then:

$$\theta_{t+1} = \theta_t + \alpha \beta_t \nabla_\theta \pi(a_t|s_t, \theta_t)$$

Interpretation 1: Gradient ascent on $\pi(a_t|s_t)$

- If $\beta_t \geq 0$: probability $\pi(a_t|s_t)$ **increases**
- If $\beta_t < 0$: probability $\pi(a_t|s_t)$ **decreases**

Proof: By Taylor expansion:

$$\pi(a_t|s_t, \theta_{t+1}) \approx \pi(a_t|s_t, \theta_t) + \alpha \beta_t \|\nabla_\theta \pi(a_t|s_t, \theta_t)\|^2$$

Exploration-Exploitation Balance

Interpretation 2: Balance between exploration and exploitation

$$\beta_t = \frac{q_t(s_t, a_t)}{\pi(a_t|s_t, \theta_t)}$$

Exploitation: β_t is proportional to $q_t(s_t, a_t)$

- If action value is large $\Rightarrow$ probability increases
- Algorithm exploits actions with high values

Exploration: β_t is inversely proportional to $\pi(a_t|s_t, \theta_t)$ (when $q_t > 0$)

- If probability is small $\Rightarrow$ enhancement is stronger
- Algorithm explores actions with low probabilities

Combined effect: The algorithm naturally balances exploration and exploitation!

Sampling Requirements

How should samples be obtained?

Sampling S:

- S should follow distribution η (either d_π or ρ_π)
- Both represent long-term behavior under π
- In practice: generate episodes following π

Sampling A:

- A should follow $\pi(A|S, \theta)$
- Select a_t following $\pi(a|s_t, \theta_t)$
- Therefore, policy gradient is **on-policy**

Practical implementation:

- Generate complete episode following current policy
- Use all samples in episode to update θ multiple times
- This is more sample-efficient than strict on-policy

Chapter Summary

Key transition: Value-based $\rightarrow$ Policy-based methods

Main concepts:

1. **Policy representation:** $\pi(a|s,\theta)$ parameterized function
    - Softmax ensures valid probabilities
    - Update by changing θ, not table entries
2. **Metrics for optimization:**
    - Average state value $\bar{v}_\pi$
    - Average reward $\bar{r}_\pi = (1-\gamma)\bar{v}_\pi$
3. **Policy Gradient Theorem:**

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\nabla_\theta \ln \pi(A|S,\theta) q_\pi(S,A)\right]$$

4. **REINFORCE algorithm:**

$$\theta \leftarrow \theta + \alpha \nabla_\theta \ln \pi(a_t|s_t,\theta) q_t(s_t,a_t)$$

Key Equations Summary

Metrics:

$$\bar{v}_\pi = \sum_s d(s) v_\pi(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1}\right]$$

$$\bar{r}_\pi = \sum_s d_\pi(s) r_\pi(s) = \lim_{n\to\infty} \frac{1}{n}\mathbb{E}\left[\sum_{t=0}^{n-1} R_{t+1}\right]$$

Policy Gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{S\sim\eta, A\sim\pi}\left[\nabla_\theta \ln \pi(A|S,\theta) q_\pi(S,A)\right]$$

REINFORCE Update:

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t,\theta_t) \sum_{k=t+1}^{T} \gamma^{k-t-1} r_k$$

Poisson Equation (undiscounted):

$$v_\pi = r_\pi - \bar{r}_\pi \mathbf{1}_n + P_\pi v_\pi$$

Comparison: Value-Based vs Policy-Based

	Value-Based	**Policy-Based**
Representation	$\hat{v}(s, w)$ or $\hat{q}(s, a, w)$	$\pi(a\|s, \theta)$
Optimal definition	Max every state value	Max scalar metric
Policy extraction	Greedy from values	Direct from π
Action space	Discrete (typically)	Discrete or continuous
Exploration	ϵ-greedy, etc.	Built-in (stochastic)
On/Off-policy	Both possible	Typically on-policy

Advantages of policy-based:

- Handles continuous action spaces naturally
- Can learn stochastic policies
- Better convergence properties in some cases

Next chapter: Actor-Critic combines both approaches!

Looking Ahead: Actor-Critic Methods

Limitation of REINFORCE:

- High variance due to Monte Carlo estimation
- Must wait until end of episode to update

Solution: Actor-Critic (Chapter 10)

- **Actor:** Policy $\pi(a|s, \theta)$ — decides which action to take
- **Critic:** Value function $\hat{v}(s, w)$ or $\hat{q}(s, a, w)$ — evaluates actions
- Use critic to estimate q_π instead of Monte Carlo
- Lower variance, can update at each step

Update rule preview:

$$\theta \leftarrow \theta + \alpha \nabla_\theta \ln \pi(a|s, \theta) \cdot \underbrace{\hat{q}(s, a, w)}_{\text{critic estimate}}$$

$$w \leftarrow w + \beta \cdot \text{TD error} \cdot \nabla_w \hat{v}(s, w)$$